

CO2 - Assessment Tool 1 -database and connectivity

CODE: DSA0506-QUERRY PROCESSING FOR DATASCIENCE

NAME: V.TEJASWINI

REG NO:192424360

1. 1. Employee and Department Management System (10 Marks)

A company maintains employee details and department information in separate PostgreSQL tables. Design **Department** and **Employee** tables using appropriate primary and foreign keys. Write a Python program to connect to the PostgreSQL database, create the tables, insert sample records, and display employee names along with their respective department names using an **SQL JOIN** query.

Sol;

```
conn = sqlite3.connect(':memory:')
```

```
cur = conn.cursor()
```

```
create_department_table_sql = """
```

```
CREATE TABLE Department(
```

```
    Department_ID INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    Department_Name VARCHAR(50)
```

```
);
```

```
"""cur.execute(create_department_table_sql)
```

```
create_employee_table_sql = """
```

```
CREATE TABLE Employee(
```

```
    Employee_ID INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    Employee_Name VARCHAR(50),
```

```
    Salary NUMERIC(10,2),
```

```
    Department_ID INT,
```

```

        FOREIGN KEY (Department_ID)

        REFERENCES Department(Department_ID)

    );

    """cur.execute(create_employee_table_sql)

    conn.commit()

    cur.close()

    conn.close()

    print("Tables 'Department' and 'Employee' created successfully in an in-memory SQLite
    database.")

```

Tables 'Department' and 'Employee' created successfully in an in-memory SQLite database.

```

import sqlite3

in all relevant cells.

create_department_table_sql = """

CREATE TABLE Department(

    Department_ID INTEGER PRIMARY KEY AUTOINCREMENT,

    Department_Name VARCHAR(50)

);

"""cur.execute(create_department_table_sql)

create_employee_table_sql = """

CREATE TABLE Employee(

    Employee_ID INTEGER PRIMARY KEY AUTOINCREMENT,

    Employee_Name VARCHAR(50),

    Salary NUMERIC(10,2),

    Department_ID INT,

```

```

        FOREIGN KEY (Department_ID)

        REFERENCES Department(Department_ID)

    );

    """cur.execute(create_employee_table_sql)

    insert_department_sql = """

    INSERT INTO Department(Department_Name)

    VALUES

    ('HR'),

    ('IT'),

    ('Finance');

    """cur.execute(insert_department_sql)

    insert_employee_sql = """

    INSERT INTO Employee(Employee_Name,Salary,Department_ID)

    VALUES

    ('Rahul',50000,1),

    ('Priya',60000,2),

    ('Anil',55000,2),

    ('Sneha',65000,3);

    """

    cur.execute(insert_employee_sql)

    conn.commit()

    cur.close()

    conn.close()

    print("Data inserted successfully into 'Department' and 'Employee' tables.")

```

```
import sqlite3

conn = sqlite3.connect(':memory:')

cur = conn.cursor()

create_department_table_sql = """

CREATE TABLE Department(

    Department_ID INTEGER PRIMARY KEY AUTOINCREMENT,

    Department_Name VARCHAR(50)

);

"""

cur.execute(create_department_table_sql)

create_employee_table_sql = """

CREATE TABLE Employee(

    Employee_ID INTEGER PRIMARY KEY AUTOINCREMENT,

    Employee_Name VARCHAR(50),

    Salary NUMERIC(10,2),

    Department_ID INT,

    FOREIGN KEY (Department_ID)

        REFERENCES Department(Department_ID)

);

"""

cur.execute(create_employee_table_sql)

insert_department_sql = """

INSERT INTO Department(Department_Name)

VALUES
```

```

('HR'),

('IT'),

('Finance');

"""

cur.execute(insert_department_sql)

insert_employee_sql = """

INSERT INTO Employee(Employee_Name,Salary,Department_ID)

VALUES

('Rahul',50000,1),

('Priya',60000,2),

('Anil',55000,2),

('Sneha',65000,3);

"""cur.execute(insert_employee_sql)

query = """

SELECT Employee.Employee_Name,

       Department.Department_Name

FROM Employee

JOIN Department

ON Employee.Department_ID = Department.Department_ID;

"""cur.execute(query)

rows = cur.fetchall()

print("Employee Name\tDepartment")

for row in rows:

    print(row[0], "\t", row[1])

```

```

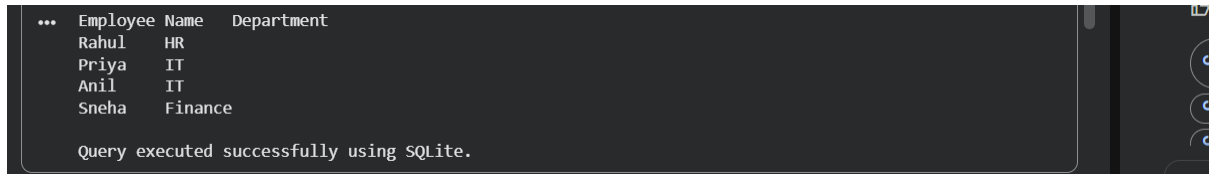
conn.commit()

cur.close()

conn.close()

print("\nQuery executed successfully using SQLite.")

```



The screenshot shows a terminal window with a dark background. It displays a table with three columns: 'Employee Name' and 'Department'. The table contains four rows of data: Rahul (HR), Priya (IT), Anil (IT), and Sneha (Finance). Below the table, a message states 'Query executed successfully using SQLite.'

Employee Name	Department
Rahul	HR
Priya	IT
Anil	IT
Sneha	Finance

Query executed successfully using SQLite.

2. Hotel Room Reservation System (10 Marks)

A hotel maintains room information and reservation details in an SQLite database. Design **Room** and **Reservation** tables using suitable primary and foreign keys. Write a Python program to connect to the database, create the tables, insert sample data, update the reservation status of a customer, and display the reservation details

```

import sqlite3

conn = sqlite3.connect("hotel.db")

cur = conn.cursor()

# Create Room table

cur.execute("""

CREATE TABLE IF NOT EXISTS Room(

Room_ID INTEGER PRIMARY KEY,

Room_Type TEXT,

Price INTEGER

)

""")

# Create Reservation table

cur.execute("""

```

```

CREATE TABLE IF NOT EXISTS Reservation(
Reservation_ID INTEGER PRIMARY KEY,
Customer_Name TEXT,
Room_ID INTEGER,
Status TEXT,
FOREIGN KEY(Room_ID) REFERENCES Room(Room_ID)
)
)

# Insert sample rooms

cur.execute("INSERT INTO Room VALUES(1,'Single',1000)")
cur.execute("INSERT INTO Room VALUES(2,'Double',2000)")
cur.execute("INSERT INTO Room VALUES(3,'Suite',3000)")

# Insert reservation

cur.execute("INSERT INTO Reservation VALUES(101,'Rahul',1,'Booked')")

# Update reservation status

cur.execute("""
UPDATE Reservation
SET Status='Confirmed'
WHERE Reservation_ID=101
""")

# Display reservation details

cur.execute("""
SELECT Reservation.Customer_Name,
Room.Room_Type,

```

```
Reservation.Status
FROM Reservation
JOIN Room
ON Reservation.Room_ID=Room.Room_ID
""")

rows = cur.fetchall()

print("Customer\tRoom\tStatus")

for row in rows:

    print(row[0], "\t", row[1], "\t", row[2])

conn.commit()

conn.close()
```

Customer	Room	Status
Rahul	Single	Confirmed